



Samueli
School of Engineering

CS-M151B

Computer Systems Architecture

Blaise Tine
UCLA Computer Science

Agenda

- Cache Exercise
- Multicore
- Final Review

Exercise: MMU with no TLB

- MMU Directly accesses L1 cache for translation
- System Setup:
 - Memory Access Time: 100 cycles
 - Address Finalization Time = 1 cycle
 - L1 Hit time = 4 cycles
 - L1 Hit rate = 80%
- **AMAT = ?**

Exercise: MMU with PIPT L1 Cache

- Physically-Indexed Physically-Tagged L1 Cache
- System Setup:
 - Memory Access Time: 100 cycles
 - Address Finalization Time = 1 cycle
 - L1 Hit time = 4 cycles
 - L1 Hit rate = 80%
 - TLB Hit time = 3 cycles
 - TLB Hit rate = 85%
- **AMAT = ?**

Exercise: MMU with VIVT L1 Cache

- Virtually-Indexed Virtually-Tagged L1 Cache
- What are the issues with this design?

Exercise: MMU with VIVT L1 Cache

- Virtually-Indexed Virtually-Tagged L1 Cache
- Challenges:
 - Homonyms
 - Different programs use same VA with different PA
 - Must flush the entire cache on context switch
 - Aliasing
 - Different VAs point to same PA
 - Consistency nightmare with duplicate entries of same PA
 - Must flush the entire cache on context switch
 - VIPT cache solves this with L1 index collision preventing dups.

Exercise: MMU with VIVT L1 Cache

- Virtually-Indexed Virtually-Tagged L1 Cache
- System Setup:
 - Memory Access Time: 100 cycles
 - Address Finalization Time = 1 cycle
 - L1 Hit time = 4 cycles
 - L1 Hit rate = 80%
 - TLB Hit time = 3 cycles
 - TLB Hit rate = 85%
- **AMAT = ?**

Exercise: MMU with VIPT L1 Cache

- Virtually-Indexed Physically-Tagged L1 Cache
- System Setup:
 - Memory Access Time: 100 cycles
 - Address Finalization Time = 1 cycle
 - L1 Hit time = 4 cycles
 - L1 Hit rate = 80%
 - TLB Hit time = 3 cycles
 - TLB Hit rate = 85%
- **AMAT = ?**

Exercise: MMU with no TLB

- MMU Directly accesses L1 cache for translation
- System Setup:
 - Memory Access Time: 100 cycles
 - Address Finalization Time = 1 cycle
 - L1 Hit time = 4 cycles
 - L1 Hit rate = 80%
- $AMAT = AMAT_{TLB} + AMAT_{L1} = (AMAT_{L1} + AFT) + AMAT_{L1}$

Exercise: MMU with PIPT L1 Cache

- Physically-Indexed Physically-Tagged L1 Cache
- System Setup:
 - Memory Access Time: 100 cycles
 - Address Finalization Time = 1 cycle
 - L1 Hit time = 4 cycles
 - L1 Hit rate = 80%
 - TLB Hit time = 3 cycles
 - TLB Hit rate = 85%
- $AMAT = AMAT_{TLB} + AMAT_{L1}$
- $= (t_{TLB} + mr_{TLB} \times AMAT_{L1} + AFT) + AMAT_{L1}$

Exercise: MMU with VIVT L1 Cache

- Virtually-Indexed Virtually-Tagged L1 Cache
- System Setup:
 - Memory Access Time: 100 cycles
 - Address Finalization Time = 1 cycle
 - L1 Hit time = 4 cycles
 - L1 Hit rate = 80%
 - TLB Hit time = 3 cycles
 - TLB Hit rate = 85%
- $AMAT = AMAT_{L1}$
- $= t_{L1} + mr_{L1} \times mp_{L1}$
- $= t_{L1} + mr_{L1} \times (TLB_access + data_access)$
- $= t_{L1} + mr_{L1} \times ((t_{LTB} + mr_{TLB} \times MemAT + AFT) + MemAT)$

Exercise: MMU with VIPT L1 Cache

- Virtually-Indexed Physically-Tagged L1 Cache
- System Setup:
 - Memory Access Time: 100 cycles
 - Address Finalization Time = 1 cycle
 - L1 Hit time = 4 cycles
 - L1 Hit rate = 80%
 - TLB Hit time = 3 cycles
 - TLB Hit rate = 85%
- $AMAT = \max(AMAT_{TLB}, t_{L1}) + mr_L1 \times mp_{L1}$
- $AMAT_{TLB} = t_{TLB} + mr_{TLB} \times mp_{TLB} + AFT$
- $mp_{TLB} = MemAT$

Exercise2: MMU with VIPT L1 Cache

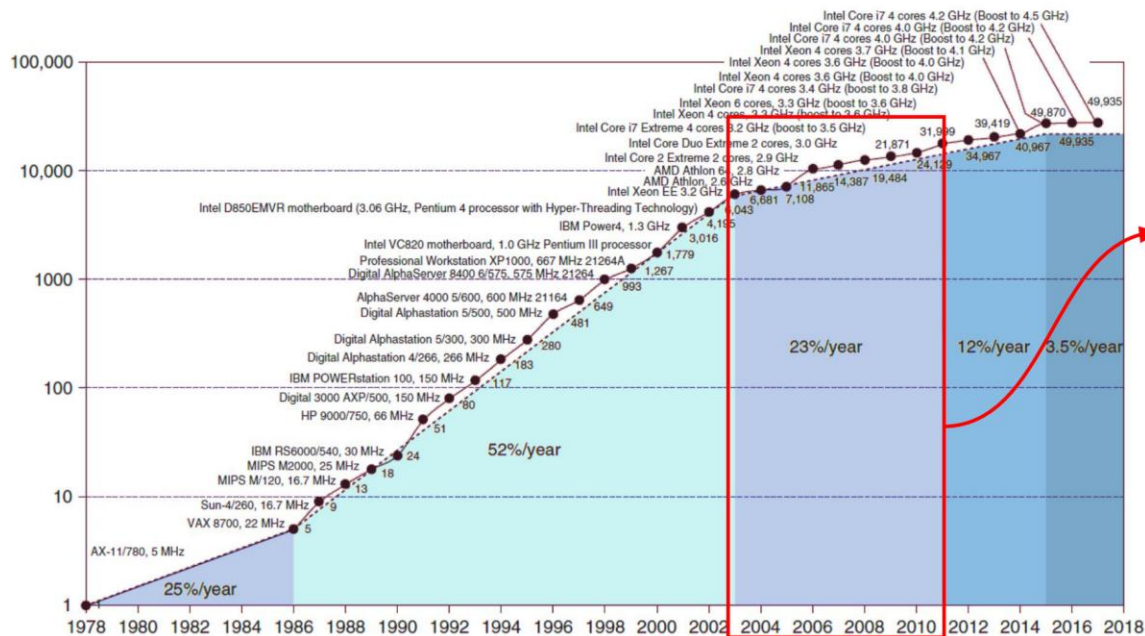
- System Setup:
 - 32-bit CPU with VM, 4KB pages, 16KB 4-way L1 cache
 - 1 GB Memory with 64B blocks
 - CPU executes:
 - I0: LW r4, 4(r1) where r1 = 0x04100
 - I1: LW r5, 4(r2) where r2 = 0x01004
 - PTE size is 4B and Page Table address = 0x80000
 - TLB contains the following VPN to PPN mappings
 - 0x00004 => 0x10000
- Questions:
 - What is the effective virtual address (VA) generated by I0 and I1?
 - Calculate L1 cache tag/index/offset for I0 and I1.
 - Determine which instruction has a TLB hit and which has a TLB miss.
 - Calculate the resulting physical address (PA) for TLB hit.
 - Calculate the PTE memory address for the TLB miss.
 - Is it possible to reduce L1 associativity to 2? Why?

Exercise2: MMU with VIPT L1 Cache

- Solution:
 - What is the effective virtual address (VA) generated by I0 and I1?
 - $I0_va=0x4104$, $I1_va=0x1008$
 - Calculate L1 cache tag/index/offset for I0 and I1.
 - $Index=\log_2(16KB/64/4)=6$, $offset=\log_2(64)=6$, $tag=PPN=(PA - 12)=30-12=18$
 - Determine which instruction has a TLB hit and which has a TLB miss.
 - $I0_VPN=0x4 \Rightarrow$ hit
 - Calculate the resulting physical address (PA) for TLB hit.
 - $I0_PPN=(0x10000 \ll 12) + 0x104 = 0x10000104$
 - Calculate the PTE memory address for the TLB miss.
 - $PTE_addr=0x8000 + (1 \times 4) = 0x8004$
 - Is it possible to reduce L1 associativity to 2? Why?
 - $16K/64/2 = 128$ sets $\Rightarrow$ 7 bits $\Rightarrow$ No because $(7+6) > page_offset$ will cause aliasing

Multicore Processors

CPU Scaling Limit



End of Dennard's scaling.
Hitting the **power wall**.

Single Core CPU Challenges

- Instruction Level Parallelism (ILP) Limitations
 - Data dependency
 - Branch prediction
 - Memory Latency

Single Core CPU Challenges

- Instruction Level Parallelism (ILP) Limitations
 - ~~Data dependency (mitigated by OoO)~~
 - ~~Branch prediction (mitigated by speculation)~~
 - ~~Memory Latency (mitigated by OoO , caches)~~
 - **Functional Units Underutilization**

Whiteboard: FU underutilized

Single Core CPU Challenges

- Increasing CPU Utilization:
 - Execute multiple programs
 - Add new hardware threads to the pipeline
 - Simultaneous multi-threading (SMT)

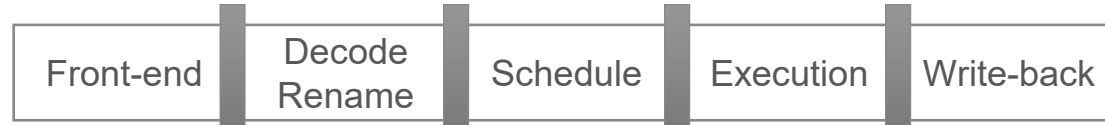
Single Core CPU Challenges

- All threads share the same functional units
- CPU Extension:
 - Each thread contains
 - PC
 - Register File
 - Execution states (CSRs)
 - Thread scheduler
- Schedule one h/w thread for execution each cycle

Whiteboard: SMT

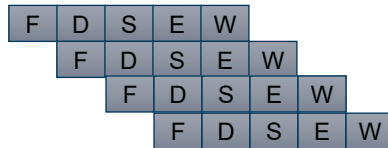
CPU pipeline

- CPU pipeline

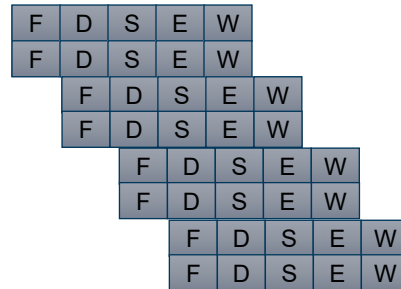


- Increasing the performance in the execution units

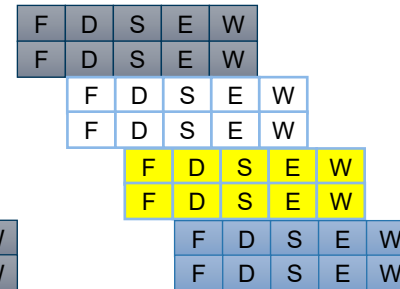
- Scalar: single issue lane.
- Superscalar: Multiple the issue lanes.
- **Multi-threading: Multiple HW threads with private PC and RegFile**



Scalar (1x PC)



Superscalar (1x PC)



Multi-threading (4x PCs)

Multi-Threading

- **Simultaneous Multi-Threading (SMT)**

- Issues instructions from multiple threads in the same cycle
- All threads share the same functional units each cycle
- Maximizes utilization of superscalar issue slots
- Example: Intel Hyper-Threading Technology

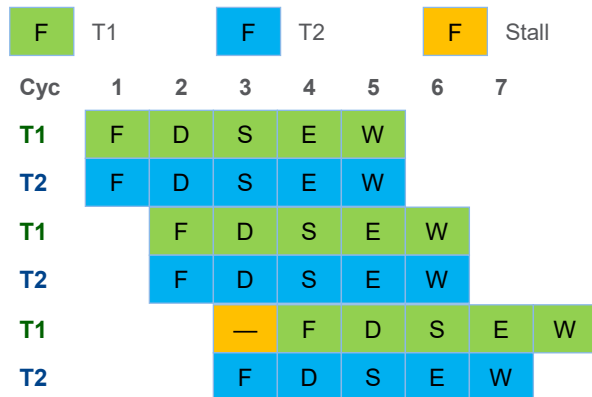
- **Fine-Grain Multi-Threading (FGMT)**

- Switches to a different thread on every cycle (round-robin)
- Hides pipeline and memory latency of stalling threads
- Only one thread issues instructions per cycle (no slot sharing)
- Simpler hardware than SMT

SMT vs. FGMT: Pipeline Visualization

Simultaneous Multi-Threading (SMT)

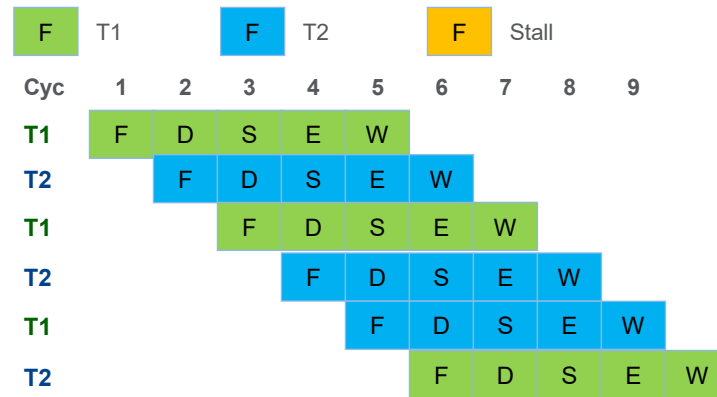
T1 and T2 issue in the same cycle



When T1 stalls, T2 is still scheduled

Fine-Grain Multi-Threading (FGMT)

T1 and T2 alternate every cycle



When T1 stalls, we switch to T2 that cycle

Single Core CPU Challenges

- Instruction Level Parallelism (ILP) Limitations
 - ~~Data dependency (mitigated by OoO)~~
 - ~~Branch prediction (mitigated by speculation)~~
 - ~~Memory Latency (mitigated by OoO , caches)~~
 - ~~Functional Units Underutilization (mitigated by SMT)~~
- **Thread-Level Parallelism Limitations**
 - FU Resource constraints
 - Cache trashing
 - Hardware scaling complexity

Exercise

- Compare a Fine-Grained Multithreaded (FGMT) and a Simultaneous Multithreaded (SMT) processor, each running two hardware threads (T0, T1). Both feature a 4-stage pipeline: Fetch (F), Issue (I), Execute (E), and Commit (C).
- Throughput: FGMT fetches and issues 1 instruction/cycle (round-robin scheduling). SMT fetches and issues up to 2 instructions/cycle total (shared dynamically across threads).
- Execution Latencies: ALU = 1 cycle. LSU (fully pipelined) = 3 cycles.
- Data Hazards: No data forwarding. Registers are written in Commit and read in Issue. Dependent instructions stall in the Issue stage until last producer commits. Register File read and Write can happen in same cycle.
- Control Hazards: Branches are fully resolved with zero stalls.
- Program (executed by both Thread 0 and Thread 1):
I1: lw x5, 0(x10) // Load data from memory
I2: add x6, x5, x7 // Data dependency on x5
- Determine the cycle-by-cycle trace for both processors
Calculate the IPC

Solution

- $IPC_{FGMT} = 4/9$, $IPC_{SMT} = 4/8$

Cycle	T0_I1	T1_I1	T0_I2	T1_I2
1	F			
2	I	F		
3	E	I	F	
4	E	E	I	F
5	E	E	-	I
6	C	E	-	-
7		C	E	-
8			C	E
9				C

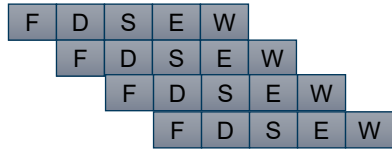
FGMT

Cycle	T0_I1	T1_I1	T0_I2	T1_I2
1	F	F		
2	I	I	F	F
3	E	E	I	I
4	E	E	-	-
5	E	E	-	-
6	C	C	-	-
7			E	E
8			C	C
9				

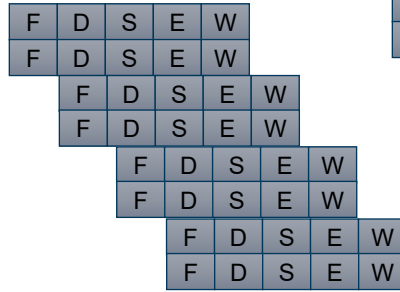
SMT

Multi-Core CPUs

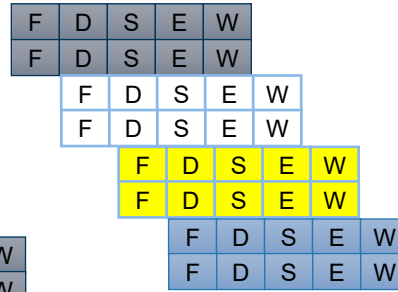
- Simply replicate the whole CPU



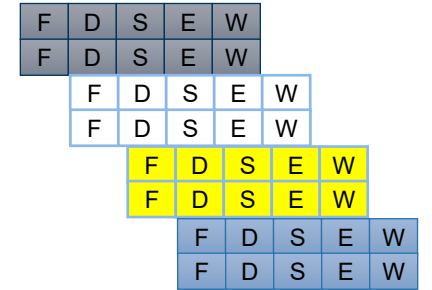
5-stage pipeline



Superscalar



Multi-threading



Multi-core

Multi-Core CPUs

- Multicore challenges
 - Cache coherence and
 - Memory consistency
 - Interconnect complexity
 - Application parallelism
 - Communication overhead
 - Balance load (Amdahl's law)

$$\text{Speedup} = \frac{1}{(1 - f) + \frac{f}{N}}$$

Where:

- f : Fraction of the task that can be parallelized (between 0 and 1).
- $(1 - f)$: Fraction of the task that is strictly serial and cannot be parallelized.
- N : Number of parallel processors (cores).

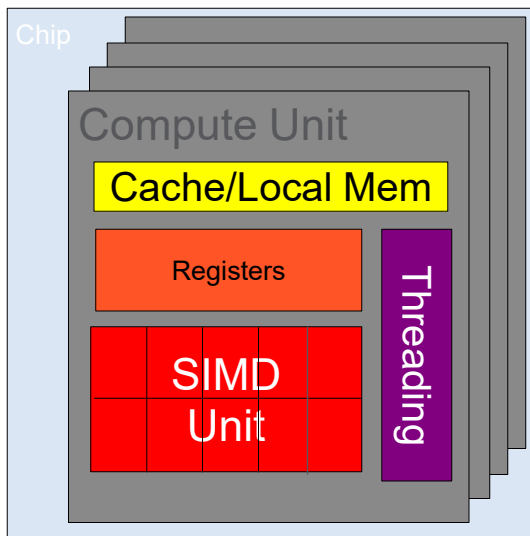
Multi Core CPUs

- Single-Instruction Multiple Data (SIMD)
 - Many applications (e.g. scientific) deal with wide data
 - Improving data-level parallelism
- Solution
 - Wider Execution Units
 - Single-floats to -> floats

CPU and GPU have different design focus

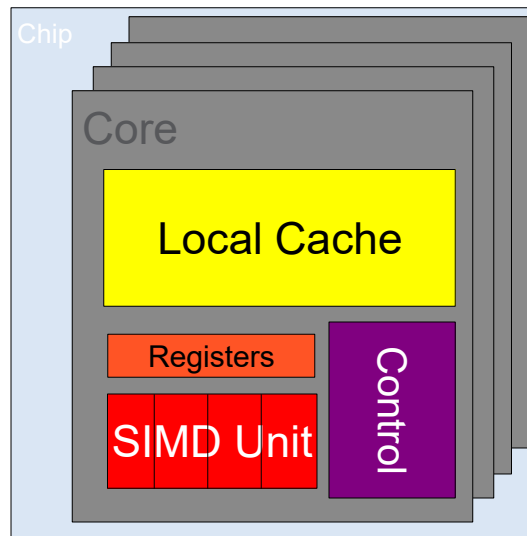
GPU

Throughput Oriented Cores



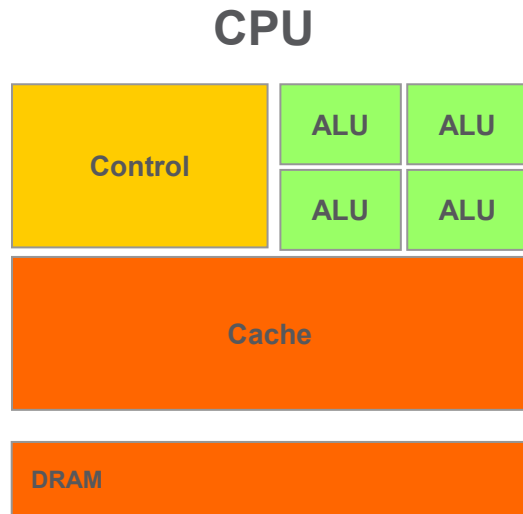
CPU

Latency Oriented Cores



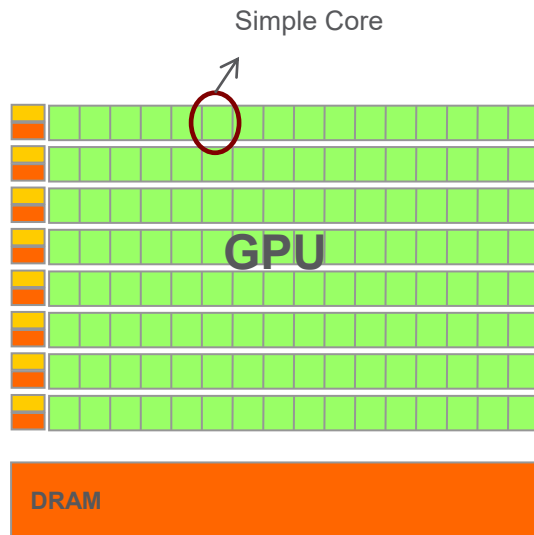
CPU: Latency-Oriented Design

- Large caches
 - Convert long latency memory accesses to short latency cache accesses
- Sophisticated control
 - Branch prediction for reduced branch latency
 - Data forwarding for reduced data latency
- Powerful ALU
 - Reduced operation latency
- Small number of hardware threads



GPU: Throughput-Oriented Design

- Small caches
 - To boost memory throughput
- Simple control
 - No branch prediction
 - No data forwarding
- Energy efficient ALUs
 - Many, long latency but heavily pipelined for high throughput
- Require **massive** number of threads to tolerate memory latencies



Attendance

- Please Fill the form with today's keyword to register your attendance.



Final Review

Final Review

- We will cover all lecture material up to today
- On paper
- You can bring notes
- No laptop but calculator allowed
- You will be provided formula for doing calculations
- The exam will mostly focus on understanding, not memorizing.

Final Review

- We will be directly tested on the following subjects
 - ISA
 - RISC-V
 - Pipelining
 - Data and control hazards
 - Single-cycle, multi-cycle, super-scalar pipelines
 - OoO pipeline (renaming, CDC6600, scoreboard, ROB)
 - Caches
 - Virtual memory
 - Power
 - Multicore

Q&A

Acknowledgement

- This course is partly inspired and use materials from the following courses created by my colleagues:
 - ECE-M1126C: Nader Sehatbakhsh @ UCLA
 - CS3220: Hyesoon Kim @ gatech